

Oppgave 2 – Tekst

Enhetstester brukes for å sikre at kode gir de forventede og ønskede resultatene. De sikrer også at ved refactoring, endringer, videreutvikling osv. at koden beholder den ønskede funksjonaliteten. De fungerer som et sikkerhetsnett, som lar utviklere være sikre på at de både lager noe som fungerer på den måten det skal, men også uten å måtte frykte at de ødelegger annen kode eller arbeid de ikke nødvendigvis har perfekt oversikt over.

Dette er spesielt viktig og nyttig når det er i en kontekst av flere utviklere som arbeider på samme prosjekt. I slike situasjoner så er det ofte at man ikke har oversikt over f.eks avhengigheter som kan finnes i andres kode. Et eksempel på dette vil være å fjerne en funksjon eller en global variabel fra sin egen kode, som man kanskje ikke lenger trenger i sitt eget arbeid, men som kan ha vært essensiell i koden til en av de andre som jobber på det samme prosjektet. Har man en god enhetstest som man følger med på regelmessig, vil man raskt kunne oppdage hvis en endring har hatt utilsiktede konsekvenser.

I oppgaven med å skrive ROT-koden så er det ikke en større gruppe jeg har måtte ta hensyn til, men den har gitt en annen form for sikkerhet. Gitt at testene jeg har skrevet fungerer som de skal, så har de gitt meg innsikt i hvor vidt koden jeg produserte gjør det den skal og om den når opp til spesifikasjonen som er gitt. Enhetstestene ga oversikt over «edge-cases» så vel som grunnleggende funksjonalitet og gjorde at det var mye enklere å planlegge hva jeg har måtte ta høyde for i koden.

Eksempelvis var jeg usikker på om koden min ville fange opp tom tekst og gi tilbake tom tekst riktig, men test nr 7 sikrer at dette faktisk er tilfellet. I et mer grunnleggende eksempel, så ga allerede de to første testene sikkerhet i at encodingen jeg hadde skrevet faktisk ga riktig output.

Jeg brukte også en del tid på å finne frem til riktig struktur for konverteringen av bokstavene. Enhetstesten som krever at store og små bokstaver returnerer i samme casing gjorde at før jeg begynte å skrive noen kode, måtte jeg planlegge om jeg skulle bruke en av flere fremgangsmåter. En av mulighetene var for eksempel å ha et eget `const capitalAlphabet` med «ABCDEFGH...osv» og så kjøre en sjekk for å finne ut om bokstaven som skulle flyttes i det lille eller store alfabetet. Jeg googlet meg frem til en litt annen sjekk i form av:

```
const isUpperCase = char !== char.toLowerCase();
```

Kilde: <https://stackoverflow.com/questions/1027224/how-can-i-test-if-a-letter-in-a-string-is-uppercase-or-lowercase-using-javascript>

Dette var da en del av researchen min før selve kodingen av `function rotateChar()` begynte, som i stedet ga meg ideen om at jeg bare kan bruke denne boolean verdien for å bestemme om en char skulle returneres i upper eller lowercase. Dette gjorde da `capitalAlphabet` unødvendig, så jeg slapp den biten med repetisjon i koden.

Det burde også nevnes at linje 92 i `logic.js` var noe jeg ble mer sikker på takket være unit testingen.

```
const index = alphabet.indexOf(lowerChar);
```

Jeg har i en annen oppgave tidligere blitt gjort klar over at `indexOf` vil returnere -1 hvis den ikke finner det man leter etter. Dette er noe jeg ikke har hatt så mye erfaring med, men det er jo en nyttig liten bit med kunnskap som i dette tilfellet lot meg sende characters, som ikke var bokstaver (så tall, utropstegn, komma, punktum osv), tilbake som det de kom inn som. Da jeg mangler mengdeerfaringen for å si med sikkerhet at denne fremgangsmetoden ville fungere riktig, så var det god trygghet i å se at flere av enhetstestene sikrer at denne fremgangsmåten faktisk gjør det den skal.